Marching Cubes Algorithm for Surface Extraction and Rendering

Luke Doladille

December 11, 2025

Introduction to Marching Cubes

A modeling algorithm's role is to generate a representation of an anatomical structure. The output can take multiple forms: probabilistic distributions, boundary representations, labels, ...

The Marching Cubes algorithm seeks to model a surface. Its input is:

- a 3D scalar field $r(x, y, z)$ discretized on a regular grid;
- an isovalue ρ.

Its output is:

- a triangle mesh that approximates the isosurface: $\{(x, y, z)|r(x, y, z) = \rho\}$

This algorithm is widely used in image-guided surgery, as it succesfully extracts surface models of bony structures, and it can be easily implemented. Here, we chose to implement the algorithm with MATLAB.

1. Importing the data

The readraw function's objective is to import the data, its dimensions, and its resolution. The input consists of two files: the Hip.vol and the Hip.raw files. These two files and the readraw function, provided as materials for the course project, are not present in the GitHub repository.

The Hip.vol file contains in the first line the size (x, y, z) of the density matrix M, and the resolution (in mms) of each (x, y, z) unit. The Hip.raw file contains the density data itself.

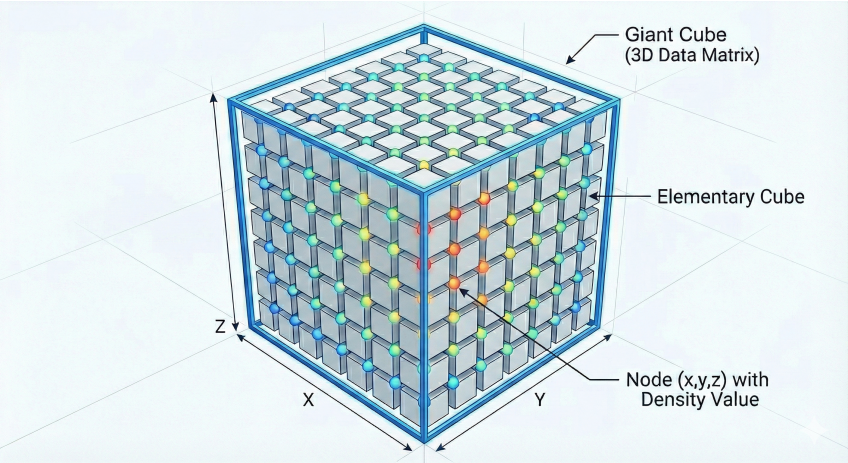


Figure 1: Density Data

2. Divide and Conquer approach

Divide-and-Conquer is a powerful algorithmic principle. Its objective is to optimize the processing of large datasets, by decomposing the processing tasks and avoiding useless computations. It is often used hand-in-hand with recursive programming.

The Initial Volume

The 3D data matrix obtained as the output of the import function readraw() contains density data. Each density value $r(x, y, z)$ corresponds to a given node. These nodes can be considered summits of elementary cubes. Therefore, the data is represented by a "giant cube" made up of small elementary cubes.

The Division part

Instead of processing each "elementary cube" sequentially, we adopt a Divide-and-Conquer Approach. This means we cut the "Giant cube" into 8 sub-cubes, displayed in blue in Fig. 2.

The Conquer part

After the division, we assess whether the isosurface passes through each of the 8 sub-cubes. This is done using a simple max/min density value check in the given sub-cube. If a cube contains a part of the isosurface, it is further subdivided, and the process repeats (cf. grey sub-cube). If a cube does not contain the surface, it is discarded.



Figure 2: Divide-and-Conquer principle

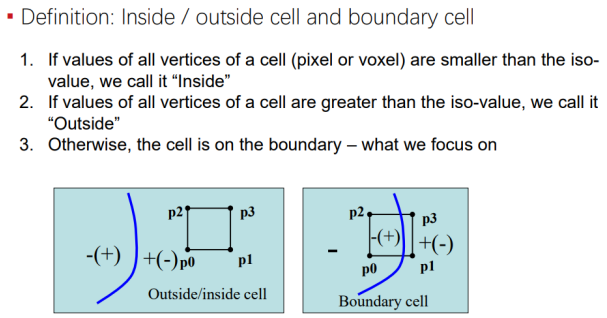


Figure 3: Defining Inside/Outside cell and Boundary cell

Implementation in MATLAB

```
% Luke Doladille, Dec. 14, 2025.

% The objective of this script is to run a recursive version of
% the Marching Cubes algorithm. The input data is the data matrix
% which contains the density values. The output is the triangle
% mesh corresponding to the boundary surfaces.

% The recursive_marching_cubes function is a wrapper function.

function [triangles, normals] = recursive_marching_cubes(data,
   isovalue)
    % data: 3D matrix of scalars
    % isovalue: Intesity threshold, in our case it represents a
       bone density
    dimX = size(data, 1);
    dimY = size(data, 2);
    dimZ = size(data, 3);
    % Start with the full volume.
    % Note: We use global indices [Start, End]
    [triangles, normals] = recurse_block(data, isovalue, [1, dimX
       ], [1, dimY], [1, dimZ]);
end

% The recursion_block() function below is the recursion block.
% It navigates in the cube by performing divide and conquer.
% It calls the process_single_cube() function, explained later.

function [tris, norms] = recurse_block(data, iso, arrX, arrY,
   arrZ)
    % data: Reference to the huge matrix (MATLAB passes by ref if
        not modified)
    % iso: The isovalue.
    % arrX: Arrays for the X axis; arrY for Y-axis; arrZ for Z-
       axis

    % 1. EXTRACT LOCAL BLOCK BOUNDS
    x1 = arrX(1); x2 = arrX(2);
    y1 = arrY(1); y2 = arrY(2);
    z1 = arrZ(1); z2 = arrZ(2);

    % 2. MIN/MAX CHECK STEP (The Conquer step)
    % We extract the sub-block values to check min/max
    sub_block = data(x1:x2, y1:y2, z1:z2);
    min_val = min(sub_block(:));
    max_val = max(sub_block(:));

    tris = [];
    norms = [];
```

```
    % If the surface does not pass through this block, reject.
    if (max_val < iso) || (min_val > iso)
        return;
    end

    % 3. BASE CASE: SINGLE CUBE
    % If the block is 1x1x1, process it.
    dx = x2 - x1;
    dy = y2 - y1;
    dz = z2 - z1;

    if (dx == 1 && dy == 1 && dz == 1)
        % We pass the top-left-front coordinate (x1, y1, z1)
        [tris, norms] = process_single_cube(data, iso, x1, y1, z1
           );
        return;
    end

    % 4. RECURSIVE STEP (The Divide step)
    % We split the longest axes.

    % Calculate Midpoints (using floor to obtain an integer)
    midX = floor((x1 + x2) / 2);
    midY = floor((y1 + y2) / 2);
    midZ = floor((z1 + z2) / 2);

    % Define ranges.
    % Left block ends at midX, Right block starts at midX.
    rangesX = { [x1, midX], [midX, x2] };
    rangesY = { [y1, midY], [midY, y2] };
    rangesZ = { [z1, midZ], [midZ, z2] };

    % Handle cases where dimension is already 1 (don't split
       further)
    if dx == 1, rangesX = { [x1, x2] }; end
    if dy == 1, rangesY = { [y1, y2] }; end
    if dz == 1, rangesZ = { [z1, z2] }; end

    % Loop through subdivisions (up to 8 children)
    for i = 1:length(rangesX)
        for j = 1:length(rangesY)
            for k = 1:length(rangesZ)
                [new_tris, new_norms] = recurse_block(data, iso,
                   rangesX{i}, rangesY{j}, rangesZ{k});
                % Append results
                tris = [tris; new_tris];
                norms = [norms; new_norms];
            end
        end
    end
end
```

Elementary cube processing

First, the elementary cube processing initializes the lists t, n.

- t is an $N \times 3$ matrix, it contains the triangle mesh which approximates the isosurface $r(x, y, z) = \rho$;
- n is a matrix containing the normal vectors of each of these triangles. These vectors indicate to MATLAB how light should be reflected off the object during the plotting phase. This matrix must have the same size as t, so it triplicates the normal vector for each triangle.

Then, the following steps are executed:

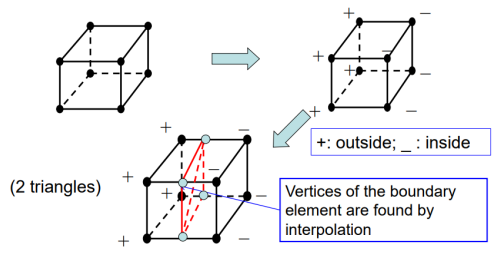
1. Retrieving the density values of each of the cube's 8 summits.
2. Defining the relative position of each summit.
3. Calculating the cube's index (cf. Fig. 4). To do this, we compute : $\sum_{i=1}^{8} \mathbb{1}(v_i > \rho) \times 2^i$. This returns a integer between 0 and 255, representing one of 2^8 configurations.
4. Calling the Look-Up Table (cf. Fig. 5). Given the configuration of the cube (its index), we want to know which of its edges intersect with the isosurface.

   Note: The Look-Up Table was copy-pasted from a GitHub repository online. It can be found here:

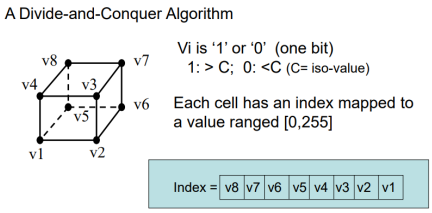
   `https://github.com/cerr/CERR/blob/master/CERR_core/PlanMetrics/heterogenity_metrics/MarchingCubes.m`. Copyright (c) 2011, Peter Hammer, All rights reserved.
5. Interpolating intersection points of the isosurface with the cube's edges.

   This step is necessary to ensure the smoothest surface possible. Indeed, if two adjacent vertices are on either side of the isosurface, this does not automatically mean that the isosurface passes through the midpoint. For example, if point 1 has a value of 10, point 2 a value of 50, and the isosurface between them is 40, using linear interpolation, we find that the isosurface should be closer to point 2 than to point 1:

   $P = P1 + \frac{\rho - V1}{V2 - V1}(P2 - P1) = P1 + \frac{40-10}{50-10}(P2 - P1) = \frac{1}{4}P1 + \frac{3}{4}P2$
6. Creating the triangles and their normal vectors.



(a) Creating a triangle mesh for the isosurface in a boundary cell



(b) Calculating the cube's index

Figure 4: Handling the intersection between the isosurface and an elementary cube

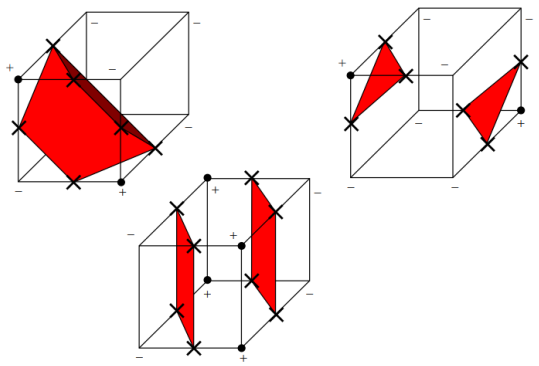


Figure 5: Finding the intersecting edges with a Look-Up Table

Implementation in MATLAB

```
function [t, n] = process_single_cube(data, iso, x, y, z)
    % Initialize t,n
    t = [];
    n = [];

    % 1. Retrieving the 8 corner values
    corners = data(x:x+1,y:y+1,z:z+1)

    % 2. Defining the relative position of each summit

    % For pos_offsets, I used the same convention we saw in class
    % This is very important, since we later use a canonical
    % matrix to look up each configuration (Look-Up Table).

    % For now, we assume that the imported  data  is oriented in
    % a similar way, with x,y,z being respectively right, up,
    % forward directions. This can be corrected later, in
    % accordance with the visualization we obtain.

    pos_offsets = [
        0 0 0;  % v1 (Bit 1)
        1 0 0;  % v2 (Bit 2)
        1 1 0;  % v3 (Bit 3)
        0 1 0;  % v4 (Bit 4)
        0 0 1;  % v5 (Bit 5)
        1 0 1;  % v6 (Bit 6)
        1 1 1;  % v7 (Bit 7)
        0 1 1   % v8 (Bit 8)
    ];

    % Mapping the previously defined corners to their scalar
    % values in the correct order.

    cube = [
        corners(1,1,1); % v1 (x,y,z) --> origin
```

```
    corners(2,1,1); % v2 (x+1,y,z) --> +1 right
    corners(2,2,1); % v3
    corners(1,2,1); % v4 (x,y+1,z) --> +1 upwards
    corners(1,1,2); % v5 (x,y,z+1) --> +1 forward
    corners(2,1,2); % v6
    corners(2,2,2); % v7
    corners(1,2,2)  % v8
];
cube = double(cube);

% 3. Calculating the cube's index (8-bits)
cube_index = 0;
for i = 1:8
    if cube(i) > iso
        cube_index = cube_index + 2^(i-1);
    end
end

% 4. Call to the Look-Up Table, to know which edges intersect
tri_table = get_tri_table();
edges = tri_table(cube_index + 1, :);

% If the cube is strictly outside or inside the surface,
   return nothing
if edges(1) == 0 %my table
    return;
end

% 5. Interpolating intersection points of surface with the
   cube's edges

% Calculating coordinates of the vertices on the given cube
vert_list = zeros(12, 3);

% Defining edge connections, represented by pairs of vertices
% e.g. : Edge 1 connects v1 and v2

edge_connections = [
    1 2; 2 3; 3 4; 4 1; % Bottom edges
    5 6; 6 7; 7 8; 8 5; % Top edges
    1 5; 2 6; 3 7; 4 8  % Vertical edges
];

% We calculate the coordinates of each intersection point
% using linear interpolation. Technically, we could optimize
% this step by only calculating the intersections
% for the intersected edges

for i = 1:12
    vA = edge_connections(i, 1); % Index of the 1st summit
    vB = edge_connections(i, 2); % Index of the 2nd summit
```

```
    % Global positions of the edge's summits (add x,y,z)
    p1 = [x, y, z] + pos_offsets(vA, :);
    p2 = [x, y, z] + pos_offsets(vB, :);

    val1 = cube(vA);
    val2 = cube(vB);

    % Linear interpolation : P = P1 + (iso - V1)(P2 - P1) / (
       V2 - V1)
    if abs(val2 - val1) > 1e-5 % Avoid division by zero
        mu = (iso - val1) / (val2 - val1);
    else
        mu = 0.5;
    end
    vert_list(i, :) = p1 + mu * (p2 - p1);
end

% 6. Triangle creation
% 'edges' contains a sequence of edges.

k = 1;
while k <= length(edges) && edges(k) ~= 0
    e1 = edges(k);
    e2 = edges(k+1);
    e3 = edges(k+2);

    % Retrieve the points calculated earlier
    p1 = vert_list(e1, :);
    p2 = vert_list(e2, :);
    p3 = vert_list(e3, :);

    % Add triangle to the list
    t = [t; p1, p2, p3];

    % Calculating a vector  normal  to the triangle
    U = p2 - p1;
    V = p3 - p1;

    normal_vec = cross(U, V);
    mag = norm(normal_vec);
    if mag > 1e-10
        normal_vec = normal_vec / mag;
    else
        normal_vec = [0, 0, 1];
    end

    n = [n; repmat(normal_vec, 3, 1)];
    k = k + 3;
end
end
```

Launch script

Here is the launch script. It contains certain adaptations for plotting purposes, such as the size of the t list and the lighting parameters.

Implementation in MATLAB

```
[data, dim, res] = readraw('Hip.vol');

[verts, norms] = recursive_marching_cubes(data, 90);

% Reshape T*9 into (3T)*3 vertex list
T = size(verts, 1);
verts_vlist = reshape(verts', 3, [])';  % Now (3T) * 3

% Close all figures
close all;
figure;

% Faces are just sequential groups of 3
num_verts = size(verts_vlist, 1);  % = 3*T
faces = reshape(1:num_verts, 3, [])';

% Apply voxel resolution to get real-world coordinates (in mm)
verts_scaled = verts_vlist .* repmat(res, size(verts_vlist,1), 1)
    ;

% Then use verts_scaled in patch
p = patch('Faces', faces, 'Vertices', verts_scaled);

% Graphic settings
set(p, 'FaceColor', 'red', 'EdgeColor', 'none');
set(p, 'VertexNormals', norms); % We apply the calculated normal
   vectors
daspect([1 1 1]); % Respect the proportions
view(3); axis tight; camlight; lighting gouraud;
grid on;
```

Results

Here are the results, found with an isovalue of 90. We can recognize a hip bone!

However, we notice that the image is flipped along the y-axis. Also, there are holes and artifacts in the isosurface. This is due to both a subpar choice for the density value and to a lacking resolution of the initial image.

Certain future improvements include the upsampling of the image used as the input, as well as determining a cost function to minimize in order to obtain an optimal isovalue.

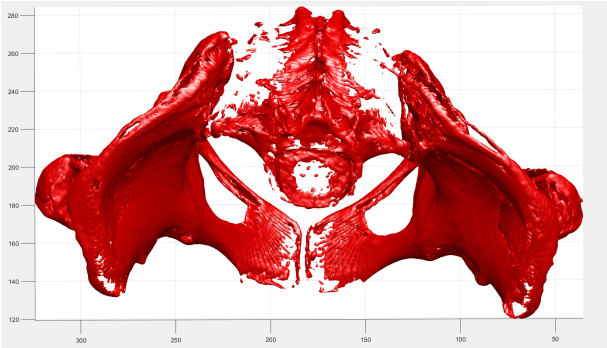


Figure 6: Projection in Oxz plane (from the top)

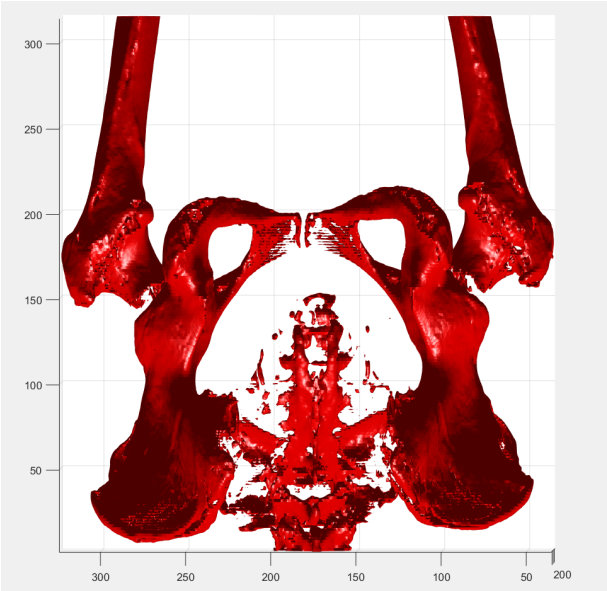


Figure 7: Projection in Oxy plane (in front)

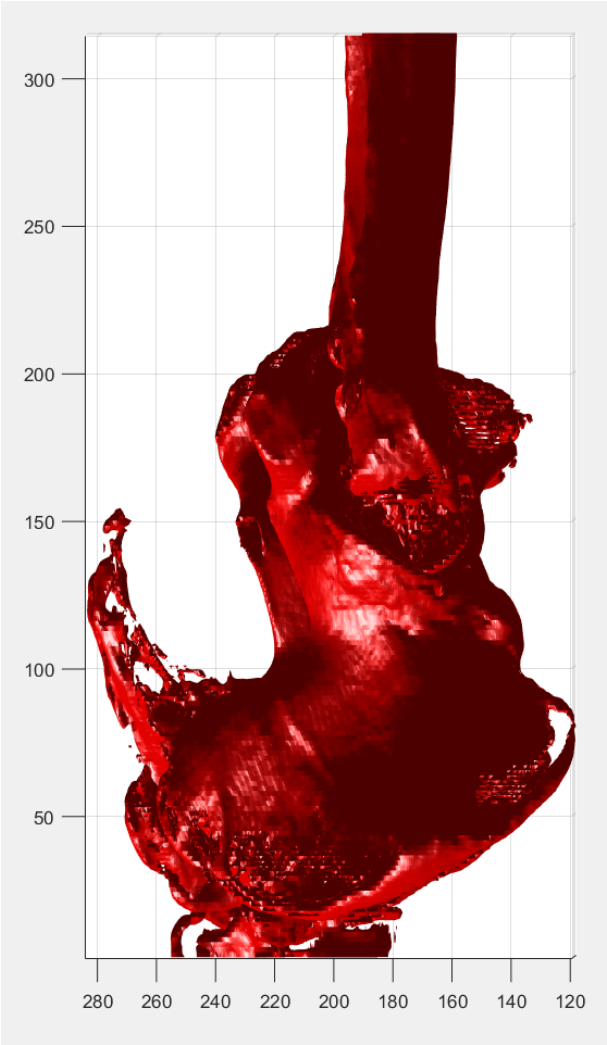


Figure 8: Projection in Oyz plane (from the side)

Sources

- Data and MATLAB readraw function were provided as project materials for course BME8113 - Advanced Intraoperative Imaging (Prof. Zheng). They are not provided in the repository.
- Figures 1 and 2 were prompted by me and AI generated by Gemini.
- Figures 3, 4a, 4b, and 5 were extracted from course materials of BME8113 - Advanced Intraoperative Imaging (Prof. Zheng).